

NoSQL Database Course Project

Migration of MovieNest Relational Database to a NoSQL Database

Introduction

The purpose of this project was to design a relational database, populate it with a large dataset, migrate the data to a NoSQL database, validate that the migration was successful, and finally visualize the migrated data using Power BI.

For this project, I used a movie rental platform called **MovieNest**, that I created during the Databases course final project in my second year. The relational database was implemented in Microsoft SQL Server, while MongoDB was chosen as the NoSQL database. Python was used to generate the dataset, perform the migration, and validate the migrated data. Finally, Power BI was used to create visualizations based on the MongoDB data.

The project demonstrates the complete data pipeline from a relational database to a NoSQL database, including data transformation and visualization.

Relational Database Design and Data Modeling

The MovieNest database was designed using a relational model consisting of six tables:

- USERS
- ACTORS
- MOVIES
- ACTORS_MOVIES
- RENTALS
- REVIEWS

The USERS table stores information about registered users, while the MOVIES table contains information about available movies. The ACTORS table stores actor information, and the ACTORS_MOVIES (the junction table between ACTORS and MOVIES) table implements the many-to-many relationship between actors and movies.

The RENTALS table stores movie rental transactions, and the REVIEWS table stores user ratings and comments for movies.

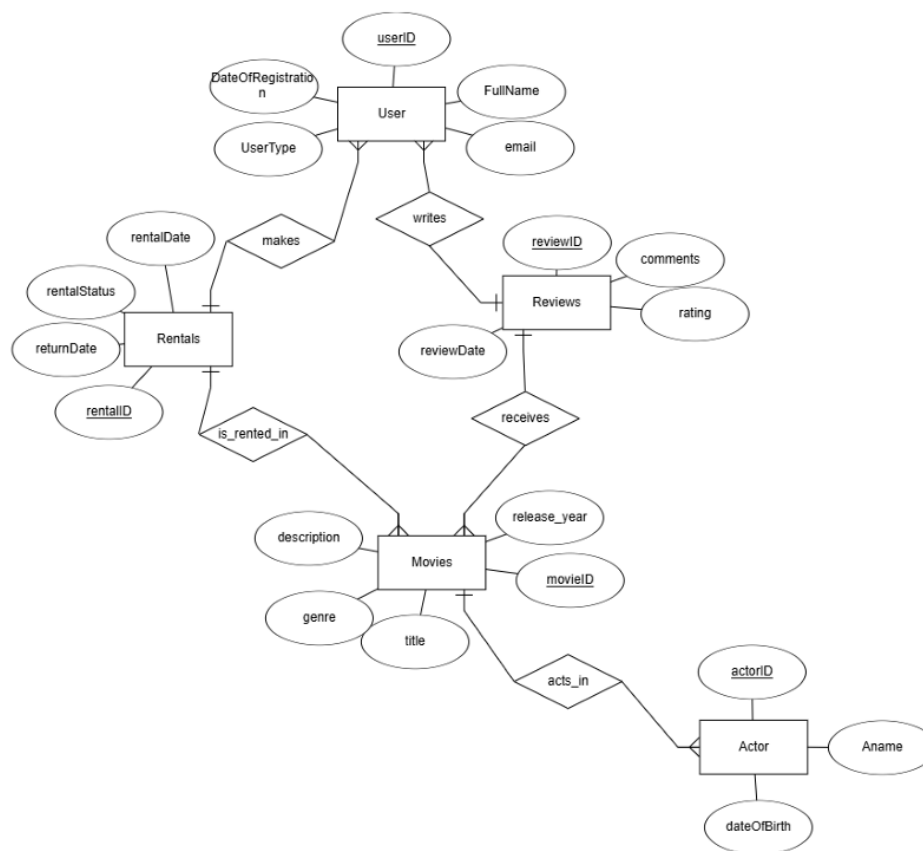
Primary keys were defined for every table, and foreign keys were used to maintain referential integrity between related tables.

I added additional constraints such as UNIQUE constraints and CHECK constraints where appropriate. For example, the email attribute is unique, and review ratings are limited to values between 1 and 5.

Simple data types such as INT, VARCHAR, DATE, and TEXT were selected because they are suitable for the stored information and easy to maintain.

To see the code for the table creation, refer to the “sql” folder inside the project folder. There you will find the file CreatingTables.sql

The ER Diagram:



Two small notes on the ER Diagram:

- The relationship between movies and actors should be many-to-many !
- Some attribute names were later changed in the code, some new ones added too.

Data Population

Instead of manually entering data, I used Python and the Faker library to generate a realistic synthetic dataset.

Table Number of Records

USERS	2,000
ACTORS	500
MOVIES	1,000
ACTORS_MOVIES	3,000
RENTALS	20,000
REVIEWS	10,000

A total of 36,500 records were generated.

The generated CSV files were imported into SQL Server using BULK INSERT commands. To see the code, refer inside the sql folder, where you will find the file DataInserts.sql.

After importing, SQL queries were executed to verify that all tables contained the expected number of records.

```
-- checking if everything is insterted
SELECT
    (SELECT COUNT(*) FROM USERS) AS UsersCount,
    (SELECT COUNT(*) FROM ACTORS) AS ActorsCount,
    (SELECT COUNT(*) FROM MOVIES) AS MoviesCount,
    (SELECT COUNT(*) FROM ACTORS_MOVIES) AS ActorMovieCount,
    (SELECT COUNT(*) FROM RENTALS) AS RentalsCount,
    (SELECT COUNT(*) FROM REVIEWS) AS ReviewsCount;
```

99 %

Results Messages

	UsersCount	ActorsCount	MoviesCount	ActorMovieCount	RentalsCount	ReviewsCount
1	2000	500	1000	3000	20000	10000

Choice of NoSQL Database

I selected MongoDB because it is one of the most widely used document-oriented NoSQL databases and is very suitable for applications where related information can be stored together inside a single document. Unlike relational databases, MongoDB allows nested documents and arrays, making it possible to reduce joins and improve query performance.

Two other NoSQL database types were also considered.

- Cassandra (Wide Column Database)

Apache Cassandra is designed for distributed systems and very large datasets. It provides excellent scalability and high availability but is more suitable for large enterprise systems than for a movie rental application.

- Neo4j (Graph Database)

Neo4j is optimized for graph relationships and network analysis. Although it can model actor and movie relationships very well, it is not ideal for storing transactional rental information and user reviews in this project.

For these reasons, I chose MongoDB because it offers a good balance between flexibility, simplicity, and performance.

NoSQL Database Modeling

The MongoDB database contains two collections:

- users
- movies

Instead of copying every SQL table directly into MongoDB, I redesigned the relational model.

The **users** collection stores user information together with two computed fields:

- totalRentals
- totalReviews

The **movies** collection stores movie information together with embedded actor information and three computed fields:

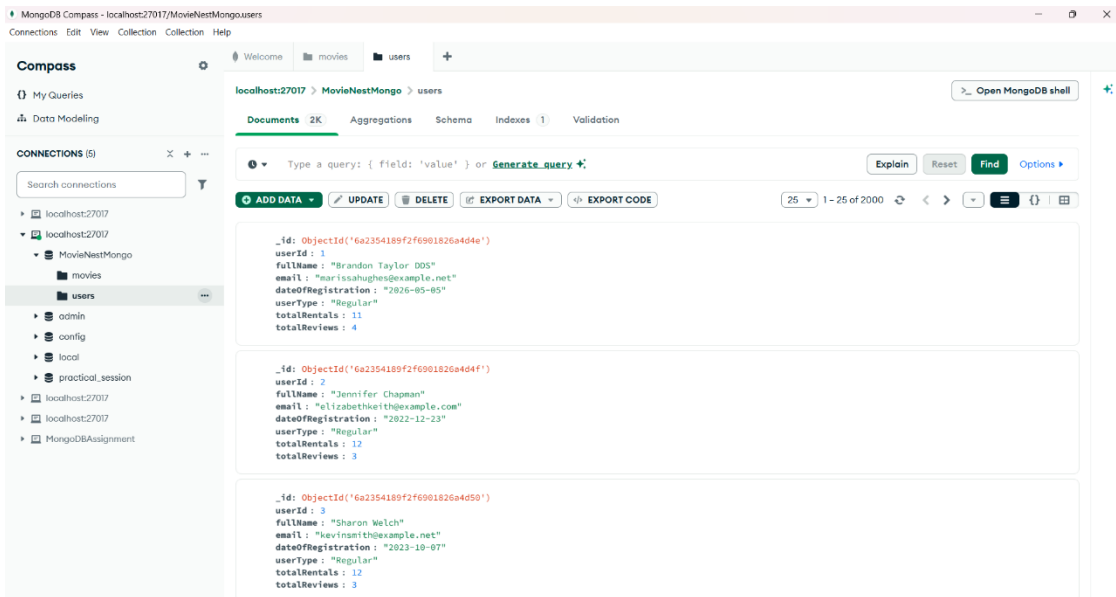
- rentalCount
- reviewCount

- averageRating

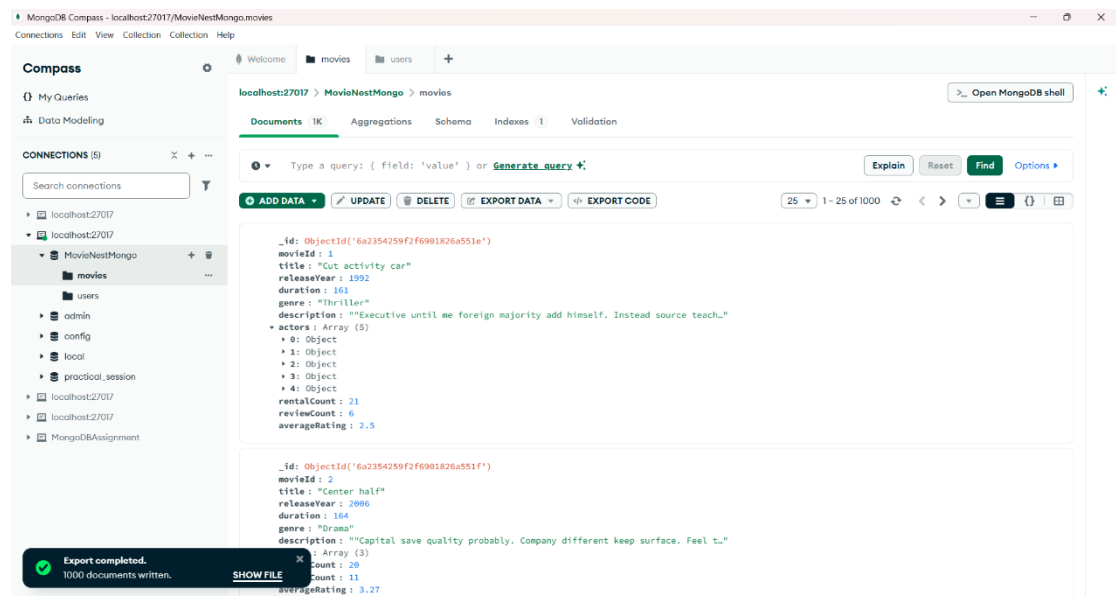
The ACTORS and ACTORS_MOVIES tables were embedded inside each movie document as an array of actors.

This denormalization removes the need for multiple joins when retrieving movie information and makes querying much faster.

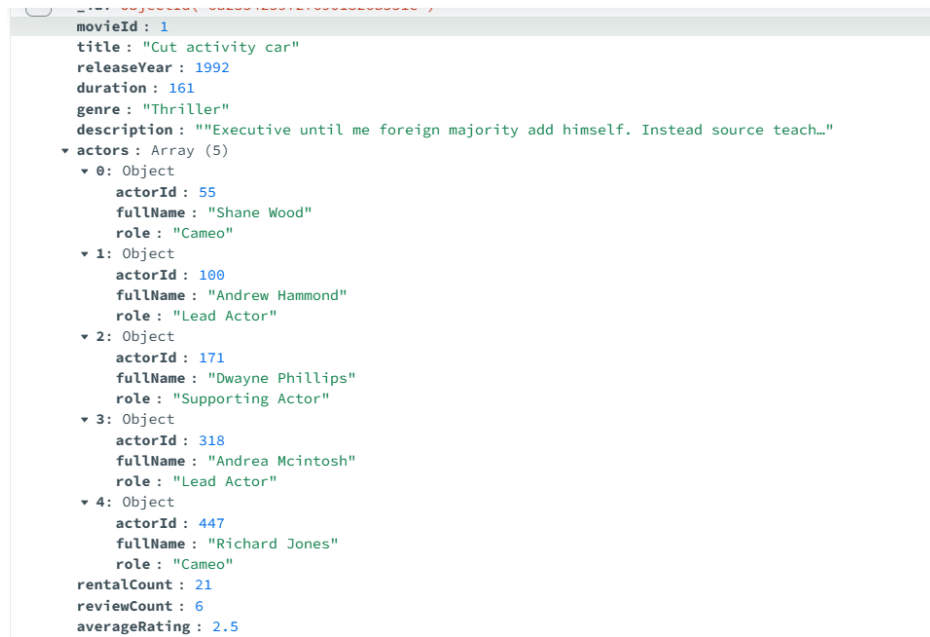
Users Collection:



Movies Collection:



Movies Document:



Data Migration Process

I implemented the migration process using Python. The migration script connects to both SQL Server and MongoDB using the pyodbs and pymongo libraries.

The migration reads relational data from SQL Server and transforms it into denormalized MongoDB documents.

Several derived fields are calculated during migration:

For movies:

- rentalCount
- reviewCount
- averageRating

For users:

- totalRentals
- totalReviews

The migration script is idempotent because existing MongoDB collections are cleared before inserting new documents. This allows the migration to be executed multiple times without creating duplicate records.

One challenge during the migration was transforming multiple SQL tables into a single MongoDB document. This was solved by embedding actor information inside each movie document and calculating aggregate values during migration.

To see the migration code, refer to the python folder, there you will find the migration.py file.

Data Validation Report

After migration, I executed an automated validation script. The script compared SQL Server and MongoDB by checking:

- Number of users
- Number of movies
- Total rentals
- Total reviews
- Top rented movie
- Average rating by genre

All validation checks returned PASS, confirming that the migration preserved the required information.

```
PS C:\Users\Dita\OneDrive\Desktop\MovieNest_NoSQL_Project\validation> python validation.py
MOVIENEST MIGRATION VALIDATION REPORT
-----
PASS - Users count matches: 2000
PASS - Movies count matches: 1000
PASS - Total rentals match: 20000
PASS - Total reviews match: 10000
PASS - Top rented movie matches: Against type how

Genre rating spot-check:
PASS - Action: 3.09
PASS - Adventure: 3.01
PASS - Comedy: 2.86
PASS - Drama: 3.06
PASS - Fantasy: 3.06
PASS - Horror: 3.02
PASS - Mystery: 3.04
PASS - Romance: 3.0
PASS - Sci-Fi: 3.0
PASS - Thriller: 2.96

Validation completed.
PS C:\Users\Dita\OneDrive\Desktop\MovieNest_NoSQL_Project\validation> |
```

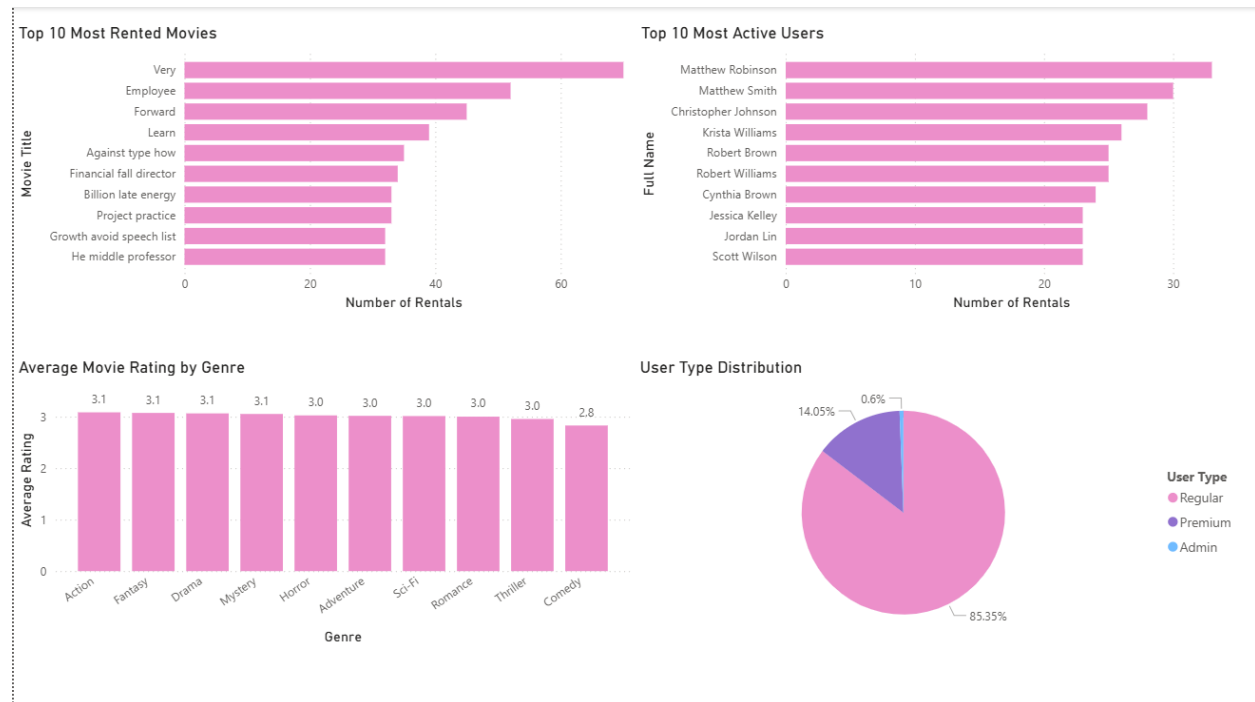
Data Visualization Layer

I used Power BI to visualize the migrated MongoDB data. I exported the MongoDB collections to CSV format and imported them into Power BI.

Four visualizations were created:

- Top 10 Most Rented Movies
- Top 10 Most Active Users
- Average Movie Rating by Genre
- User Type Distribution

These visualizations demonstrate the usefulness of the denormalized MongoDB model and make use of the derived fields calculated during migration.



Conclusion

This project successfully demonstrated the complete migration of a relational SQL Server database to MongoDB. The project included relational database design, synthetic data generation, migration with data transformation, automated validation, and visualization using Power BI. The migration process showed how relational tables can be redesigned into document-based collections using denormalization and embedded documents. Working on

this project improved my understanding of SQL databases, NoSQL databases, data migration techniques, Python scripting, and business intelligence visualization tools.

Overall, the project demonstrates an end-to-end data engineering workflow and highlights the advantages of MongoDB for storing denormalized document data.

Worked on by: Dita Jusufi

ID: 131713

GitHub: <https://github.com/ditjusufi>